A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution

Jeff Dean, David Patterson, and Cliff Young
Google Brain

The end of Moore's law and Dennard scaling has led to the end of rapid improvement in general-purpose program performance. Machine learning (ML), and in particular deep learning, is an attractive alternative for architects to explore. It has recently revolutionized vision, speech, language understanding, and many other fields, and it promises to help with the grand challenges facing our society. The computation at its core is low-precision linear algebra. Thus, ML is both broad enough to apply to many domains and narrow enough to benefit from domain-specific architectures, such as Google's Tensor Processing Unit (TPU). Moreover, the growth in demand for ML computing exceeds Moore's law at its peak, just as it is fading. Hence, ML experts and computer architects must work together to design the computing systems required to deliver on the potential of ML. This article offers motivation, suggestions, and warnings to computer architects on how to best contribute to the ML revolution.

As scientists and engineers, it's our responsibility to address society's biggest problems, such as those in Figure 1. Many of the items on that list might seem to be beyond the skillset of computer scientists and engineers, but happily, that is not the case. We believe advances in ML will lead to significant advances on most of these grand challenges.

The deep-neural-network branch of ML is a transformational technology that, in the last five years, has started revolutions in several fields. In 2012, image recognition didn't work well. AlexNet[1] broke previous accuracy records of the ImageNet competition[2] and has led to a sequence of further improvements such that machines now exceed human accuracy in image-recognition tasks.[3–6] Similar breakthroughs have accelerated fields such as speech recognition,[7] web search,[8] human language translation,[9–10] medical diagnosis,[11–13] and even playing the game

Published by the IEEE Computer Society
0272-1732/18/$33.00 ©2018 IEEE

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

of Go.[14] We expect this trend of breakthroughs to continue and broaden to provide new avenues of research and development for today's grand challenges.

Make solar energy affordable	Reverse-engineer the brain	Provide energy from fusion	Prevent nuclear terror
Develop carbon sequestration methods	Secure cyberspace	Manage the nitrogen cycle	Enhance virtual reality
Provide access to clean water	Advance personalized learning	Restore and improve urban infrastructure	Engineer the tools for scientific discovery
Advance health informatics	Engineer better medicines	Enable universal communication	Build flexible general-purpose AI systems

Figure 1. Societal grand challenges. The National Academy of Engineering proposed the first 14 in 2008.[15] A decade later, the authors added two new challenges regarding universal communication and AI systems, which seem appropriate for today.

The current ML revolution requires two kinds of scale: in the datasets that are available and the computing resources used to analyze them. Large datasets contain the raw material to understand the world around us. Thus far, large-scale ML computing has been done in large datacenters containing hordes of GPUs, which were originally designed to accelerate graphics. We are transitioning to an era where datacenters will be filled with computers designed solely for ML computations. They will come not just from GPU manufacturers, but also from startups, from new product lines by traditional microprocessor suppliers, and from more established Internet companies that had not previously built their own processors.

For example, Google began designing custom ML ASICs in 2013 and has deployed them in its datacenters since 2015. Our first-generation TPU[16] (shown in Figure 2) focused on the production phase of ML, because deploying ML models at Google scale demanded unprecedented computing power. We projected that deep neural networks would become so popular that they would double computation demands on our datacenters (as indeed, they have). The production phase is called inference or prediction. The earlier development phase is called training or learning. The second-generation Cloud TPU[17] (shown in Figure 3), deployed in 2017, trains large ML models at scale quickly. Both datacenters and edge devices (such as cars, phones, and watches) have custom hardware for inference, but the majority of training cycles will likely reside in datacenters.

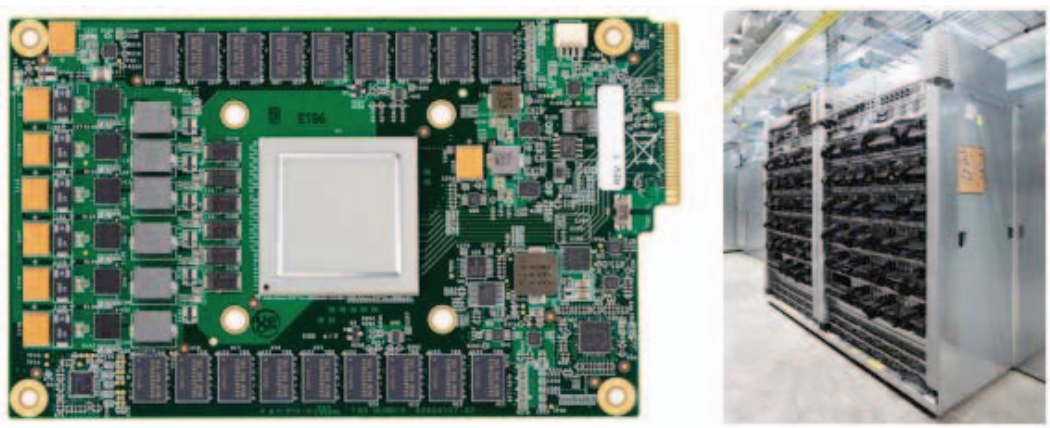

Figure 2. The first-generation Google TPU chip, board (left) and system (right). AlphaGo ran on the TPUs in these racks to defeat Lee Sedol at Go in 2016.

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

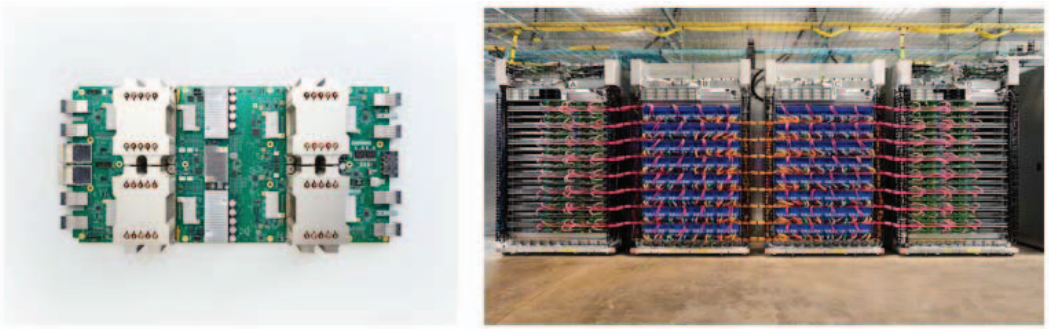

Figure 3. The second-generation Google TPU board (left) and system (right). The board has four chips. The collection of racks includes 64 boards and a custom high-speed network.

Even with these new custom accelerators, our ML researchers remain hungry for more. We observe that no matter what hardware is available, ML researchers have a "patience threshold" that limits how long they are willing to wait for the results of an experiment (much as computer architects have a threshold for how long they are willing to wait for simulations to run). The threshold varies between a few hours and a week, depending on the researcher. Faster hardware enables larger, more ambitious experiments within the confines of this patience threshold; smaller experiments run the risk that they won't explore a large-enough space to improve ML accuracy. What this means for computer systems designers is that demand for ML cycles is effectively unbounded. Speeding up model evaluation by any amount is valuable; factors of 10 or 100 would likely increase the rate of recent breakthroughs such as those mentioned above.

Alas, just when ML provides a burst of seemingly inexhaustible demand for computing cycles, computer architecture is nearing the end of Moore's law and Dennard scaling. Their demise raises the stakes even higher for system designers—we need to solve exponentially harder problems without the exponentially more resources that Moore's law used to deliver. We will need to find better, more efficient solutions at the same time as we tackle larger, more difficult problems.

ADVICE FOR HARDWARE ENGINEERS

ML applications span the entire spectrum of hardware design points, from the datacenter and supercomputers, to automotive applications, mobile devices, and the Internet of Things. Each of these design points comes with a different package of constraints on cost, power, size, and the mixture of compute, communication, and storage required for a solution.

Moreover, ML algorithms are changing rapidly. Figure 4 shows the number of ML articles posted on arXiv.org per year. They grow faster than the Moore's law rate of doubling every two years. More than 50 ML articles now appear daily on arXiv alone! The good news is that these algorithms are commonly expressed in higher-level frameworks, such as TensorFlow,[18] Caffe,[19] or PyTorch,[20] which raise the programming abstraction and make it easier to map the program to innovative architectures.

Hardware designs must remain relevant over at least two years of design time plus a three-year deployment window (assuming standard depreciation schedules). As designers, we need to pick a good system-level balance point between specialization for the currently popular techniques and flexibility to handle changes in the field over the lifetime of a design. Designing appropriate hardware for a five-year window in a field that is changing as rapidly as ML is quite challenging. Compilation techniques, possibly themselves enhanced by ML, will be central to meeting this challenge. It would be ideal if users could express their ML problems in the most natural way, but then rely on compilation to give interactive response times for researchers and scalability for production users.

Because of this wide range of constraints, it is hard to imagine that a single solution is the best overall, in the same way that the x86 architecture came to dominate general-purpose computa-

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

tion in the late 1990s. The end of Moore's law might also mean an end to the "trickle-down" effect in computer architecture, where this year's high-end part can be better integrated and cost-reduced to be the mainstream part of two years from now, and the embedded part of four years from now.

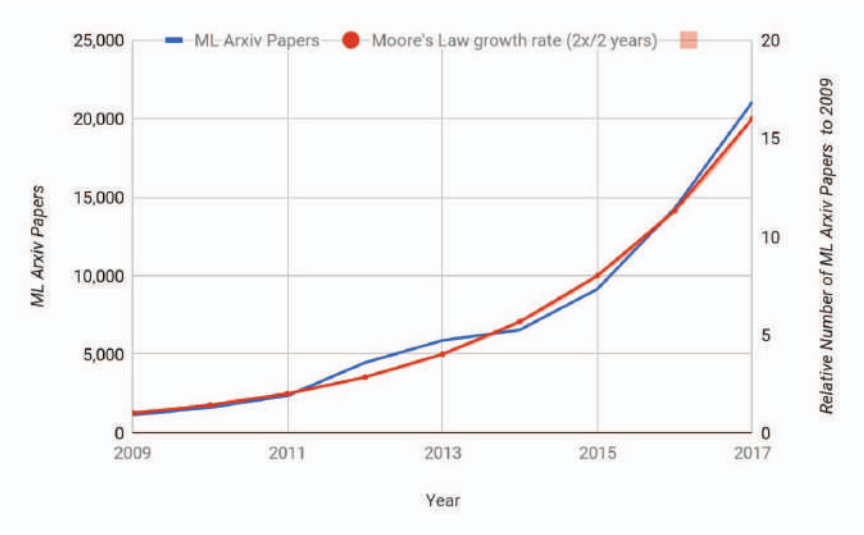

Figure 4. ML arXiv articles per year. They have more than doubled every two years, which is more than Moore's prediction for chips in 1975. We counted articles in the cs.CV, cs.CL, cs.LG, cs.AI, cs.NE, and stat.ML topics.

This section discusses six issues that impact ML hardware design, roughly sorted on a spectrum from purely architectural to mostly ML-driven concerns.

Training

Both training and inference are important, but insufficient computer architectural work is going into training. Inference is easier, not only because it takes one-third as many arithmetic operations as training. Storage requirements are larger in training because activation values must be saved for back-propagation, where the feedback about the model's prediction accuracy is applied to update the model's parameters. Assuming good-enough latency, inference workloads can be scaled out. Training doesn't necessarily scale out—it currently requires many expensive, sequential steps. It's not surprising that many companies' (including Google's) first forays into deep-learning hardware have aimed at inference first. Tackling inference is only solving the easier, more scalable piece of the end-to-end problem, however.

Batch Size

Batch size[21] has fundamental implications for architecture—it enables an important form of operand reuse—but it is surprisingly poorly understood, especially for training. The perceived knowledge in the deep-learning field says that the maximum accuracy improvement per computational step comes from stochastic gradient descent with momentum at a minibatch size of 1. Today's GPUs and TPUs operate efficiently at minibatch sizes of 32 or larger, raising questions about whether architectures for minibatch = 1 might be more effective. Empirical results are confusing; a recent sequence of articles[22–23] has shown that image-oriented convolutional models can train effectively at minibatch sizes of 8,192 and 3,2768. We are unaware of similar results for multi-layer perceptron classifiers or for LSTM-based models. If batch size could be made

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

arbitrarily large while still training effectively, then training is amenable to standard weak scaling approaches. However, if the training rate of some models is restricted to small batch sizes, then we will need to find other algorithmic and architectural approaches to their acceleration.

Sparsity and Embeddings

Sparsity in ML takes many forms, which lend themselves to very different approaches to hardware support. Many recent architectural articles address fine-grain sparsity, exploiting zeros and small values to reduce work. And indeed, because of the use of rectified linear units (ReLU)[24] as activation functions, many models exhibit significant levels of fine-grained sparsity in their activation values. However, we think that coarse-grain sparsity, where an example touches only a fraction of the parameters of huge model, has even more potential; Mixture of Experts (MoE) models[25] consult a learned subset of a panel of experts as part of their network structure. Thus, MoE models train more weights using fewer flops for higher accuracy than previous approaches. Embeddings, which transform huge sparse spaces (such as vocabularies) into more compact dense representations suitable for linear algebra operations, have received relatively little attention from the architecture community, yet they are key to textual applications like web search and translation. Unlike other kinds of accesses to parameters in neural networks, accesses to embedding tables often involve many relatively small, random accesses in very large data structures (hundreds of 100- to 1,000-byte reads in multi-hundred-gigabyte data structures per training or inference example).

Quantization and Distillation

A number of techniques have been deployed to great effect to provide cost-effective inference, but might also greatly benefit training. Quantized or reduced-precision arithmetic representations[26] have proven themselves in several inference accelerators and are now available in GPUs. Sadly, there is very little work on training in reduced-precision arithmetic, and the little published work focuses on toy-sized problems like MNIST and CIFAR-10 (see the pitfall in the next section). Distillation[27] uses a larger model to bootstrap the training of a smaller model, while achieving higher accuracy than directly training the smaller model on the same inputs. It's not clear why this is the case. Could better training methods allow us to directly train the smaller models (and perhaps all models) to higher accuracy? Is there something fundamental about the more degrees of freedom in the larger model that enables better training?

Networks with Soft Memory

Some deep-learning techniques provide functionality akin to random-access memory; doing so is complicated by preserving differentiability so the techniques can support back-propagation. Examples include Neural Turing Machines,[28–29] memory networks,[30] and attention.[31] Attention allows Google Translate to map from a word in the target language to the corresponding word or phrase in the source language. Such soft mechanisms are expensive compared to traditional "hard" memories, because the soft memory computes a weighted average over all entries of a table. A hard memory simply loads a single entry from a table. We haven't seen research into efficient or sparse implementations of these soft memory models.

Learning to Learn (L2L)

Recent results from Zoph et al.[32] suggest that neural-network models can themselves search for and develop better neural-network models. They used the CIFAR-10 image recognition competition to show that L2L could match the accuracy of the best models developed by ML experts. The L2L search used the smaller CIFAR-10 dataset instead of ImageNet to make the computational workload plausible on today's hardware, and nevertheless required hundreds of GPU cards for a week. Despite huge growth in the field, there is a shortage of experts on deep-learning technologies. L2L offers the possibility that we might use far more computing resources but require considerably less human ML expertise in designing ML solutions, which seems like a utilitarian

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

optimization of marginal returns on capital and labor. For those doing hardware-software co-design, L2L gives the further possibility to construct better systems by simultaneously, automatically searching for new ML model architectures and new computer architectures.

ML HARDWARE FALLACIES AND PITFALLS

We complement the six issues above with four warnings, following the "fallacies and pitfalls" style.[33]

Fallacy: Given the large size of the ML problems, the hardware focus should be operations per second (throughput) rather than time to solution (latency).

You need to know the right target if you are going to hit it. Proper metrics are important to understand and apply. Because it is easy to collect, many ML benchmarks compare input examples per second (throughput) when measuring the performance of ML systems. The original TPU article[16] pointed out that latency is even more important than throughput for user-facing inference workloads. The best measure of a training system is time to converged accuracy, or the wall-clock time to fully train a model to a desired accuracy.

Fallacy: Given a sufficiently large speedup, ML researchers would be willing to sacrifice a little accuracy.

Take great care when trading accuracy for performance. It's sometimes okay, but it depends a lot on specifics. Using a grand challenge from Figure 1, loss of accuracy on "preventing nuclear terror" is probably not good. For many ML researchers, a high-throughput training system that loses 0.5 percent in accuracy is uninteresting. At the other extreme, some mobile applications accept higher accuracy losses to fit on-device.

Pitfall: Designing hardware using last year's models.

Figure 4 suggests the rapid rate of change in ML. Recent architecture articles for ML have evaluated their proposals using MNIST, CIFAR-10, and AlexNet. While MNIST is important historically, it is 20 years old and useful primarily for student programming assignments. CIFAR-10 was the image-recognition competition that ImageNet replaced in 2010 because it became too easy. Even AlexNet, which kickstarted the CNN revolution by winning ImageNet in 2012, is now old fashioned. It has only six neural-network layers, while the 2017 champion has 150. Furthermore, it has the bulk of its parameters in two fully connected layers at the top of the model, a characteristic that is missing from nearly every other accurate image model since AlexNet. Designing hardware or evaluating proposed optimizations based on MNIST, CIFAR-10, or AlexNet is no more effective or persuasive than designing general-purpose processors evaluated with quicksort, hashing, or binary search. Ideally, the field would develop benchmark suites for deep-learning accelerators, similarly to the way that workstation manufacturers developed the SPEC (http://spec.org/) benchmarks. Until such benchmarks emerge, using the recent winners of ML competitions such as ImageNet ensures up-to-date, if narrow, results.

Pitfall: Designing ML hardware assuming the ML software is untouchable.

In the heydays of Moore's law and ample instruction-level parallelism, an architect could contribute without understanding what the program was trying to do. In fact, the ground rules of benchmark suites like SPEC2006 disallowed changes to the source code, so studying it didn't help. A hardware team that takes the time to learn some of the constraints and paradigms of the

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

ML community is much more likely to produce an effective solution than one that sticks to familiar territory. There is no requirement to run the identical model; a faster but different computation with the same accuracy is a welcome result. The goal is a hardware/software system that more effectively solves the problem.

CONCLUSION: A GRAND OPPORTUNITY

ML advances in the next decade might help solve some of the grand challenges in Figure 1. For example, a car that never crashes was science fiction in 2000, but thanks to advances in ML, might be a reality in 2020. These past successes and exciting future prospects have accelerated the demand for compute cycles for ML.

In the past, specialized hardware rarely made sense, because it often applied to narrow pieces of our overall problems. However, ML hardware is an exception; we can build specialized machines uniquely tailored to it—very fast, low-precision linear algebra—and then apply them to a huge and growing swath of the world's computing needs going forward: vision, speech, language understanding, and so on. Given that performance of general-purpose code on traditional microprocessors has plateaued along with Moore's law and Dennard scaling, perhaps domain-specific architectures for ML should be the next major hardware focus. ML applies to a vast and growing range of useful computations, and exploration of the hardware design space for ML has only just begun.

Given that performance of general-purpose code on traditional microprocessors has plateaued along with Moore's law and Dennard scaling, perhaps domain-specific architectures for ML should be the next major hardware focus.

One critical shortage in the race to roll out exciting applications of ML is availability of ML experts. L2L systems might answer this shortage. Suppose we wanted to run L2L on a big problem instead of CIFAR-10. L2L trained about 14,000 experimental models, each of which took about one hour on one GPU for CIFAR-10. If we instead trained 100,000 experimental models (because the more complex problem needs more data points to learn from), and each such experiment required 200 GPUs to train for a week (which is the size of some of the translation models trained today), the total is 20-M GPU weeks. That is, doing L2L to develop a state-of-the-art model on a problem of significant scale might require 100,000 times more computation than what our earliest L2L experiments on CIFAR-10 used.

From an architect's viewpoint, the next decade promises to be exciting. The demand for ML computing is growing much faster than Moore's law, just when Moore's law itself is fading. Architects need to deliver innovative machines to empower ML experts, and together, we will be able to address the grand challenges that our society faces.

ACKNOWLEDGMENTS

We thank the Hot Chips and *IEEE Micro* reviewers for their excellent suggestions. Our colleagues in Google's Brain and Platforms teams have helped us immensely in understanding and mapping these co-design problems.

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

1. A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25* (NIPS), 2012; https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.
2. O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, no. 3, 2015, pp. 211–252; https://dl.acm.org/citation.cfm?id=2846559.
3. C. Szegedy et al., *Going Deeper with Convolutions,* arXiv.org, 2014; https://arxiv.org/abs/1409.4842.
4. S. Ioffe and C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,* arXiv.org, 2015; https://arxiv.org/abs/1502.03167.
5. C. Szegedy et al., *Rethinking the Inception Architecture for Computer Vision*, arXiv.org, 2015; https://arxiv.org/abs/1512.00567.
6. K. He et al., *Deep Residual Learning for Image Recognition*, arXiv.org, 2015; https://arxiv.org/abs/1512.03385.
7. G. Hinton et al., "Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups," *IEEE Signal Processing Magazine*, vol. 29, no. 6, 2012, pp. 82–97; http://ieeexplore.ieee.org/document/6296526/.
8. J. Clark, "Google Turning Its Lucrative Web Search Over to AI Machines," *Bloomberg Technology*, 2015; www.bloomberg.com/news/articles/2015-10-26/google-turning-its-lucrative-web-search-over-to-ai-machines.
9. Y. Wu et al., *Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation*, arXiv.org, 2016; https://arxiv.org/abs/1609.08144.
10. M. Johnson et al., *Google's Multilingual Neural Machine Translation System: Enabling Zero-Shot Translation*, arXiv.org, 2016; https://arxiv.org/abs/1611.04558.
11. V. Gulshan, L. Peng, and M. Coram, "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," *JAMA*, vol. 316, no. 22, 2016, pp. 2402–2410; http://jamanetwork.com/journals/jama/fullarticle/2588763.
12. Y. Liu et al., *Detecting Cancer Metastases on Gigapixel Pathology Images*, arXiv.org, 2017; https://arxiv.org/abs/1703.02442.
13. J. Olczak et al., "Artificial intelligence for analyzing orthopedic trauma radiographs," *Acta Orthopaedica*, 2017; www.tandfonline.com/doi/full/10.1080/17453674.2017.1344459.
14. D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, 2016, pp. 484–489; www.nature.com/nature/journal/v529/n7587/full/nature16961.html.
15. *14 Grand Challenges for Engineering in the 21st Century*; www.engineeringchallenges.org/challenges.aspx.
16. N. Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit," *Proc. 44th Annual International Symposium on Computer Architecture* (ISCA), 2017, pp. 1–12; https://dl.acm.org/citation.cfm?id=3080246.
17. *Cloud TPUs: Google's second-generation Tensor Processing Unit is coming to Cloud*, Google.ai, 2017; http://g.co/tpu.
18. M. Abadi et al., *Tensorflow: Large-scale machine learning on heterogeneous distributed systems*, arXiv.org, 2016; https://arxiv.org/abs/1603.04467.
19. Y. Jia et al., "Caffe: Convolutional Architecture for Fast Feature Embedding," *Proceedings of the 22nd ACM international conference on Multimedia* (MM), 2014, pp. 675–678; https://dl.acm.org/citation.cfm?id=2654889.
20. A. Paszke and S. Chintala, *PyTorch*, 2017; www.pytorch.org.
21. *Machine Learning Glossary*, Google; https://developers.google.com/machine-learning/glossary/#batch size.
22. P. Goyal et al., *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*, arXiv.org, 2017; https://arxiv.org/abs/1706.02677.
23. Y. You et al., *100-epoch ImageNet Training with AlexNet in 24 Minutes*, arXiv.org, 2017; https://arxiv.org/pdf/1709.05011.pdf.

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.

24. M.D. Zeiler et al., "On rectified linear units for speech processing," *Proc. IEEE Int'l Conf on Acoustics, Speech and Signal Processing* (ICASSP), 2013, pp. 3517–3521; https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/40811.pdf.
25. M. Shazeer et al., *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer*, arXiv.org, 2017; https://arxiv.org/abs/1701.06538.
26. P. Warden, *What I've learned about neural network quantization*, 2017; https://petewarden.com/2017/06/22/what-ive-learned-about-neural-network-quantization/.
27. G. Hinton, O. Vinyals, and J. Dean, *Distilling the knowledge in a neural network*, arXiv.org, 2015; https://arxiv.org/abs/1503.02531; doi.org/arXiv:1503.02531.
28. A. Graves, G. Wayne, and I. Danihelka, *Neural Turing machines*, arXiv.org, 2014; https://arxiv.org/abs/1410.5401; doi.org/arXiv:1410.5401.
29. A. Graves et al., "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, 2016, pp. 471–476; www.nature.com/articles/nature20101.
30. J. Weston, S. Chopra, and A. Bordes, *Memory Networks*, arXiv.org, 2014; https://arxiv.org/abs/1410.3916.
31. D. Bahdanau, K. Cho, and Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, arXiv.org, 2014; https://arxiv.org/abs/1409.0473.
32. B. Zoph and Q. Le, *Neural Architecture Search with Reinforcement Learning*, arXiv.org, 2016; https://arxiv.org/abs/1611.01578.
33. J. Hennessy and D.A. Patterson, *Computer Architecture: a Quantitative Approach*, Elsevier, 2018; www.elsevier.com/books/computer-architecture/patterson/978-0-12-811905-1.

ABOUT THE AUTHORS

Jeff Dean is a Google Senior Fellow in the Research Group, where he co-founded and leads the Google Brain project, Google's deep-learning and AI research team. His research interests include large-scale distributed systems, machine learning, performance monitoring, compression techniques, information retrieval, microprocessor architecture, and compiler optimizations. He has a PhD in computer science from the University of Washington. Contact him at jeff@google.com.

David Patterson is a Distinguished Engineer in the Google Brain project, a professor at the University of California, Berkeley, and the vice-chair of the RISC-V Foundation's board of directors. He is the co-author of the book "Computer Architecture: A Quantitative Approach." His primary research interest is domain-specific computer architectures for machine learning and in open instruction set architectures. He has a PhD from the University of California, Los Angeles. Contact him at davidpatterson@google.com.

Cliff Young is a researcher in the Google Brain project, where he works on co-design for deep-learning accelerators. He is one of the designers of Google's TPU. His research interests include machine intelligence, software systems, special-purpose computer architecture, and optimizing compilers. He has a PhD in computer science from Harvard University. Contact him at cliffy@google.com.

Authorized licensed use limited to: PUC MG. Downloaded on May 26,2021 at 09:15:12 UTC from IEEE Xplore. Restrictions apply.